

CSE 4403: Algorithms

Lecture12

Dynamic Programming - I

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

What even is dynamic programming?

Dynamic Programming (DP) sounds intimidating. It's not. At its heart, it's one simple idea:

CORE IDEA

Don't solve the same sub-problem twice. Break a big problem into smaller pieces, solve each piece once, remember the answer, and reuse it.

That's it. Everything else — tables, recurrences, optimal substructure — are just details of *how* to do this efficiently.

The Staircase Analogy

Imagine you're counting how many ways you can climb a 10-step staircase, taking 1 or 2 steps at a time.

To reach step 10, you came from step 9 (1 step) or step 8 (2 steps). So:

RECURRENCE

$$\text{ways}(10) = \text{ways}(9) + \text{ways}(8)$$

This works for any step. And once you've computed $\text{ways}(9)$, you never need to recompute it. That's DP.

Two Necessary Ingredients

INGREDIENT 1

Overlapping Subproblems

The same smaller problem appears many times. Without this, DP gives no benefit over plain recursion.

INGREDIENT 2

Optimal Substructure

The best solution to a big problem is built from best solutions of smaller problems.

Signs a problem might need DP

- Count the number of ways
- Find minimum/maximum
- Make decisions repeatedly
- **Problem can be broken into smaller versions of itself**

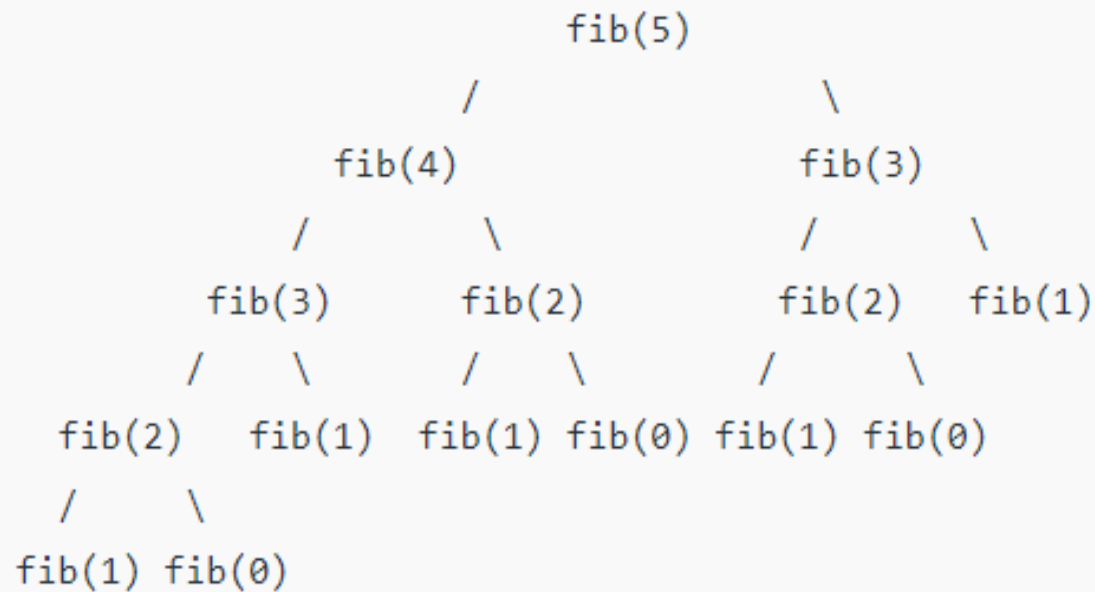
Problem	DP?
Fibonacci	✓
Climbing Stairs	✓
Shortest Path (DAG)	✓
Binary Search	X
Merge Sort	X

The Classic Example: Fibonacci

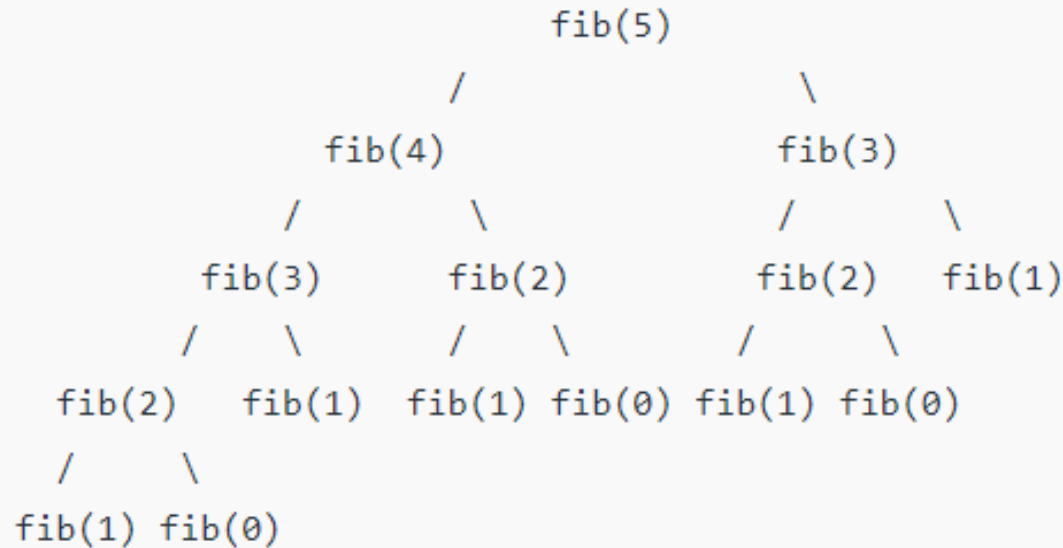
Fibonacci numbers: 1, 1, 2, 3, 5, 8, 13, 21 ... where $F(n) = F(n-1) + F(n-2)$.

```
int fib(int n){  
    if(n<=1) return n;  
    return fib(n-1)+fib(n-2);  
}
```

The Classic Example: Fibonacci



The Classic Example: Fibonacci



THE PROBLEM

`fib(5)` calls `fib(3)` twice. `fib(10)` calls `fib(3)` **eight times**. `fib(40)` makes over **300 million** calls. The time complexity is $O(2^n)$ — exponential.

Fix 1 — Memoization

(Top-Down DP)

```
int fib(int n){  
    if(n<=1) return n;  
  
    if(dp[n]!=-1)  
        return dp[n];  
  
    return dp[n]=fib(n-1)+fib(n-2);  
}
```

RESULT

Each value is computed **exactly once**. Time complexity drops from $O(2^n)$ to **$O(n)$** . fib(1000) is now instant.

Memoization Table — Step by Step

(Top-Down DP)

Fibonacci memoization table							
n=0	n=1	n=2	n=3	n=4	n=5	n=6	n=7
-1	-1	-1	-1	-1	-1	-1	-1

Memoization Table — Step by Step

(Top-Down DP)

Base case: fib(0) = 0, fib(1) = 1							
n=0	n=1	n=2	n=3	n=4	n=5	n=6	n=7
0	1	-1	-1	-1	-1	-1	-1

Memoization Table — Step by Step

(Top-Down DP)

$\text{fib}(2) = \text{fib}(1) + \text{fib}(0) = 1 + 0 = 1 \rightarrow \text{saved in memo}$

n=0	n=1	n=2	n=3	n=4	n=5	n=6	n=7
0	1	1	-1	-1	-1	-1	-1

Memoization Table — Step by Step

(Top-Down DP)

$\text{fib}(3) = \text{fib}(2) + \text{fib}(1) = 1 + 1 = 2 \rightarrow \text{saved in memo}$

n=0	n=1	n=2	n=3	n=4	n=5	n=6	n=7
0	1	1	2	-1	-1	-1	-1

Memoization Table — Step by Step

(Top-Down DP)

$\text{fib}(7) = \text{fib}(6) + \text{fib}(5) = 8 + 5 = 13$ ✓ Done!

n=0	n=1	n=2	n=3	n=4	n=5	n=6	n=7
0	1	1	2	3	5	8	13

Fix 2 — Tabulation

(Bottom-Up DP)

```
dp[0]=0;  
dp[1]=1;  
  
for(int i=2;i<=n;i++)  
    dp[i]=dp[i-1]+dp[i-2];
```

The Two Approaches

(Bottom-Up DP)

MEMOIZATION

Top-Down

Start from the big problem, recurse, cache results. More intuitive. Can skip unneeded subproblems.

TABULATION

Bottom-Up

Start from base cases, fill a table forward. No recursion overhead. Often faster in practice.

Both give the same answer and time complexity. Tabulation is often preferred in competitive programming for its simplicity and avoiding stack overflow.

The DP Recipe

How to solve a DP problem

- Define the state
- Define the recurrence
 - Means: How to combine the previous results to get current?
- Identify base cases
- Compute/store answers
- Return the desired state

Example (Fibonacci):

- State: $dp[i]$ = i th Fibonacci number
- Recurrence: $dp[i] = dp[i-1] + dp[i-2]$
- Base: $dp[0] = 0$, $dp[1] = 1$
- Answer: $dp[n]$

Time Complexity

(Bottom-Up/Top-Down DP)

Fill-up the DP table by calculating each value only once
 N = Size of the DP table

Total Time needed:

N * Time required combine the pre-calculated results

For Fibonacci Problem: Combining takes constant time. (Addition)

So overall Complexity: $O(N)$

Space Optimization

(Bottom-Up DP)

```
dp[0]=0;  
dp[1]=1;  
  
for(int i=2;i<=n;i++)  
    dp[i]=dp[i-1]+dp[i-2];
```

```
a=0;  
b=1;  
  
for(...)  
{  
    c=a+b;  
    a=b;  
    b=c;  
}
```

Time and Space Complexity

Method	Time	Space
Recursion	$O(2^n)$	$O(n)$
Memoization	$O(n)$	$O(n)$
Tabulation	$O(n)$	$O(n)$
Optimized	$O(n)$	$O(1)$

Climbing Stairs

Problem:

You can climb 1, 2, or 3 steps at a time. How many ways to reach step n ?

```
dp[i] = ways to reach i
```

Find the state.

Find the recurrence relation.

Find the base cases.

...

Home Task

Problem:

- You have coins: 1, 3, 4
- Find the minimum number of coins needed for spending **6 units**.

Solve using DP:

Find the state.

Find the recurrence relation.

Find the base cases.

...